Lab 3: Traffic Light Controller

Part 1 due February 26, 2021
Part 2 due March 12, 2021

1 Assignment

For this assignment, you'll be designing, synthesizing, and uploading to your FPGA dev-board a controller for the traffic light system of a 4-way intersection. The intersection has a North-South road, an East-West road, and pedestrian cross walks.

Introduction

All previous labs have only required combinational logic—the wires and gates you have been using were stateless elements that implemented logical circuits. In this project, we will focus on **sequential logic**, where elements (e.g., flip-flops) can encode "internal" state. *Stateful* elements are crucial to most computations and correspond roughly to "variables" in programming languages.

> You may now use all features of SystemVerilog, including `always` blocks, control logic (`if`/`else`, `case` statements), and any arithmetic operators and equality operators (so long as they synthesize).
>
> See Section 5 and the SystemVerilog course reader for more guidance on sequential circuits and finite state machines (FSMs). Many new constructs, including non-blocking assignment, will be necessary for this lab. Be sure to understand these new concepts before starting.

Retrieving Handout

> Please click this link to clone the Lab 3 handout code repository.

Modules

In `tlc_top.sv` you will see 3 modules instantiated which you need to implement in their respective `*.sv` files:

1. `clk_divider`,
2. `timer`,
3. `tlc` (traffic light controller)

as well as the logic interfacing the modules and the onboard switches and LEDs. You need not pay attention to the `hex_to_sseg` or `disp_mux` modules instantiated (they are used to display the countdown output of your `timer`).

1.1 Clock Divider

We want our `tlc` (traffic light controller) module and `timer` to operate at a frequency of 1Hz, so that we can operate in units of seconds (not fractions of a second). Given a clock of 100MHz, we'll need to make an adjustment.

> In `clk_divider.sv`, complete the implementation of a clock divider that meets the following requirements:
>
> - The clock divider takes a clock (`clk_in`)—your 100MHz system clock—and outputs a 1MHz signal (`clk_out`) to be used elsewhere as a clock enable.
> - The output clock enable (`clk_out`) should only remain high for **one** input clock period (from the 100MHz clock).
> - The clock divider takes a reset (`rst`) input which resets the divider to a state in which it has seen 0 positive edges from the clock input (`clk_in`).

In other words, for every 100 million positive edges of the input clock (`clk_in`), there should be one high cycle of the output clock enable (`clk_out`). Section 1.2 motivates why the short duty-cycle (on-time : off-time ratio).

Using your FPGA

The red reset button onboard will reset the clock divider (i.e., serve as its `rst` input signal).

1.2 Timer

We require different states of our traffic light controller to last for a customized duration, so we need a dynamically adjustable timer.

> In `timer.sv`, complete the implementation of a timer module that meets the following requirements:
>
> - The timer produces a 4-bit (unsigned) output (`out`) that represents the current time remaining.
> - The timer takes a reset (`rst`) input, which when asserted, should reset the timer to its maximum value.
> - The timer takes an initial value (`init`) and a `load` signal. When `load` is asserted, the current time remaining should get set to the initial value (`init`).
> - The timer takes a clock (`clk`)—your 100MHz system clock—and a clock enable (`clk_en`) input.
> - The timer takes an enable (`en`) input, and when `en` is asserted, the timer should decrement by 1 on every new clock enable, but it should stop at 0. If the timer enable (`en`) is not asserted, the timer should stay at its current value (unless a `load` or `rst` occurs).

`timer_rst` should take highest precedence, followed by `load`, and then `en`. The output should change at most once per `clk_en` which motivates *why* the clock divider needed to produce an output which is high for only one 100MHz clock period—as your timer's flip flops are still **only** clocked by the 100MHz system clock (in their sensitivity list) to prevent clock-related timing bugs.

Using your FPGA

The rightmost digit on the seven-segment display will show (in hexadecimal) the output of your timer (i.e., current time remaining). Similarly, `led[3:0]` will show the same information.

The "down" button will reset the timer. (We use different reset buttons because of the different frequencies at which the clock divider and timer modules operate.)

The center button will be used as your `load` signal input, and `sw[3:0]` will set the initial value (`init`) for your timer (from which it will count down).

1.3 Traffic Light Controller

The traffic light controller (`tlc`) takes a 100MHz clock (`clk`) and the previously created 1Hz clock enable signal (`clk_en`)—which assists in setting the frequency at which this unit should operate—as well as a reset `rst` input.

The traffic light controller interfaces with the `timer` module via `tlc` outputs of `timer_rst`, `timer_en`, `timer_init`, and `timer_load` used to control the timer. Then, as an input to `tlc`, the traffic light controller takes `timer_out`, the timer's output, to resolve when time has elapsed for a given state (as counted down by the timer module).

Finally, the controller takes `car_ns`, `car_ew`, and `ped` to indicate the presence of a car on either road or a pedestrian, and the controller should output `led_ns`, `led_ew`, and `led_ped` to turn on the appropriate "traffic lights."

Complete the implementation of `traffic_light_controller.sv` that meets the following requirements:

1. The traffic light should reset to an idle state in which both lights are red and both pedestrian walk signals are off.
2. The idle state should go immediately to the pedestrian state, in which both lights are red, and both pedestrian signals are on.
3. The pedestrian state should last for 15 seconds before transitioning to the next state.
4. If there is a car on the North-South road and not on the East-West road, the next state from the pedestrian state should have the North-South light green and the East-West light red.
5. If there is a car on the East-West road and not on the North-South road, the next state should have the East-West light green and the North-South light red.
6. If there is a car on both roads or neither road, the pedestrian state should go to the road that was red most recently (i.e., if the North-South was green more recently and there are no cars, East-West should be green next).

7. After reset, the pedestrian state should lead to green on the North-South road.
8. Green lights should last for 10 seconds and should immediately transition to yellow lights on the same road.
9. Yellow lights should last for 5 seconds.
10. Following a yellow light, if there is a pedestrian, the controller should proceed to the pedestrian state. Otherwise, it should go to green on the other road.
11. When one traffic light is green or yellow, the other should be red.
12. When one traffic light is green or yellow, the same pedestrian light should be on, and the opposite pedestrian light should be off.

Hint: The staff solution uses multiple FSM states for each combination of lights listed above. Depending on your implementation, it could be a good idea to have an initialization state to set the timer to the proper count and an execution state in which the timer is enabled. Additionally, it is acceptable if your controller takes an additional cycle after the timer reaches 0 before either the light changes and/or the timer is reloaded.

Using your FPGA

The LEDs on your board will be used to represent the "traffic lights" and pedestrian walk-signals as follows:

- `led[7:5]` will represent the red, yellow, and green lights for the North-South road, respectively
- `led[4]` will represent the North-South pedestrian walk signal
- `led[3]` will represent the East-West pedestrian walk signal
- `led[2:0]` will represent the red, yellow, and green lights for the East-West road, respectively

Similarly, the switches will represent the presence of cars or pedestrians as follows:

- any of `sw[7:5]` will represent a car waiting on the North-South road
- either of `sw[4:3]` will represent a pedestrian waiting to cross the street
- any of `sw[2:0]` will represent a car waiting on the East-West road

We will continue to use 2 different reset buttons—the red reset button to reset the clock divider and the "down" button for the traffic light controller and the timer—because of the different frequencies at which the modules operate.

Note: In `tlc_top.sv`, the lines pertaining to I/O for the first clock divider and timer deliverable (see comment `PART 1 I/O`) should be removed to allow the onboard `led` array to act as the traffic lights and the traffic light controller `tlc` to control the timer's enable (`en`), `load`, and initial value (`init`) signals.

2 Deliverable

Start Part 2 early as the workload is not necessarily divided evenly across the two deliverables.

Part 1 (Due February 26, 2021)

> Design and implement the clock divider and timer. Create testbenches for each module, and draft a short writeup of your testing methodology. For this part, you should also submit an initial FSM state-transition diagram for the traffic light controller `tlc` module.

Part 2 (Due March 12, 2021)

> Design and implement the traffic light controller `tlc` module. Create a testbench and write a description of your testing strategies for the controller. Update your FSM diagram to reflect any new changes.

Submission

For each part (both weeks), submit the following via GitHub Classroom:

1. HDL Code (keep your code organized and modular by having one module for each source file), synthesized bitstream and utilization report, and our handout files (constraint file, Tcl scripts).
2. A description (not more than a few typed paragraphs) of your testing methodology (simulation) for each part.
3. A state-transition diagram for the FSM which underlies your traffic light controller `tlc`—an initial draft for Part 1 and the final form for Part 2.

Part 1 Submission

Because this is a two part, two week lab, we will use the same repository and GitHub Classroom assignment for both portions. **For Part 1 only**, place your ***grading*** commit on the `main` branch of the repository auto-generated when you accepted the GitHub Classroom assignment repository ***with*** the following commit message: `PART 1 SUBMISSION`

Then submit via Gradescope to the correponding assignment (Lab 3.1). **Add your lab partner** as a group member, and remember that updates aren't auto-pulled in Gradescope (you *may* need to manually resubmit if you make changes after your initial submission).

For Part 2, submit via Gradescope to the corresponding, separate assignment (Lab 3.2). We expect that this submission will match your latest commit to the `main` branch of the repository auto-generated when you accepted the GitHub Classroom assignment and joined a team. **Add your lab partner** as a group member, and remember that updates aren't auto-pulled in Gradescope.

Your repository should have the following structure:

```
<LAB3_REPO_NAME>/
    constraint/
        Nexys_A7.xdc
    hdl/
        *.sv
    tcl/
        *.tcl
    synth_output/
        tlc.bit
        ...
        post_route_util.rpt
    fsm_diagram.png or fsm_diagram.pdf
    description.txt or description.pdf
```

Specifically, **your submission should contain**:

- Your SystemVerilog files
- Your synthesized bitstream
- A post-route utilization report
- The provided constraint file
- The provided Tcl scripts
- Circuit sketches
- Testing methodology description

If you have any questions, please come to office hours or post on Piazza.

3 (Tentative) Rubric

Component	Part 1 Points	Part 2 Points
FSM diagram	5	10
Simulation (testing) description	5	5
Testbench rigor (see below)	10	15
Correct Clock Divider Implementation	15	
Timer Implementation	15	
Traffic Light Controller Implementation		30
Student code has no warnings	5	5

Reminder: This is a tentative rubric subject to change. Additionally, a few additional points are reserved for proper adherence to our GitHub Classroom repository submission guidelines enumerated above.

Updated February 23, 2021, Dhilan Ramaprasad, Jon Cruz